PROJECT DOCUMENTATION

FitBuddy

AI Fitness Plan Generator using Gemini Models

Submitted by

S.kaviya (Leader)

P.deepika

K.yuvasri

M.santhiya

D.yugamathi

Department: B.Sc Computer Science

College: Prof. Dhanapalan College of Science and Management

Academic Year: 2025 - 2028

Acknowledgement

We would like to thank our college management, our head of the department and our project guide for their support and guidance during this project. We also thank our friends and family for testing the website and giving us useful suggestions. Finally, we thank all the team members for working together to finish FitBuddy.

Abstract

FitBuddy is a web application that makes a personal 7-day workout plan for a user with the help of Google Gemini AI models. The user creates an account and enters age, weight, fitness goal and workout intensity. The server sends these details to Gemini 2.5 Pro, which returns a day-by-day plan with warm-up, main exercises and cooldown. Gemini 2.5 Flash gives a short nutrition and recovery tip and also powers a private diet chat. If the user is not happy with the plan, they can write feedback and the AI prepares an updated plan. The project is built with Python and FastAPI, uses SQLAlchemy with SQLite or PostgreSQL, and is deployed on Render. Security features such as password hashing, email OTP, mobile OTP and bot protection are included.

1. Introduction

1.1 Overview

Many people want to start exercising but do not know which exercises to do, how many sets to do, or how to change the routine when it becomes too easy or too hard. A personal trainer is the best solution, but it is costly and not everyone can afford it. FitBuddy tries to solve this by using an AI model as a virtual coach that is available all the time.

1.2 Problem Statement

General workout plans found on the internet are not made for a particular person. They do not consider age, weight or the goal of the person, and they cannot be changed easily after giving feedback. There is a need for a simple website that creates a personal plan quickly and updates it when the user needs a change.

1.3 Objectives

- To generate a personal 7-day workout plan using Gemini models.
- To give a nutrition and recovery tip along with the plan.
- To update the plan based on the user's feedback.
- To provide a private diet chat for general meal ideas.
- To keep user accounts secure using password hashing, OTP and bot checks.
- To deploy the project online on Render.

1.4 Scope

FitBuddy is meant for beginners and normal gym users who want a starting guide. It is not a medical tool. The plans and diet ideas are general suggestions, and a person with any health problem should consult a doctor or a qualified trainer before following them.

2. System Study

2.1 Existing System

Right now people depend on trainers, YouTube videos, fitness magazines or fixed plans in mobile apps. Trainers are costly, videos are not personal, and fixed plans do not change based on what the user feels after doing the workouts.

2.2 Proposed System

The proposed system collects the basic profile of the user and uses Gemini to create the plan. The plan and the feedback are stored in the database, so the user can log in later and see them again. The system also has a fallback plan, so the website still works if the AI service is not available.

2.3 Advantages

- Personal plan in a few seconds, available at any time.
- Plan can be changed by typing feedback in normal language.
- No extra cost for a trainer.
- Works in mobile browsers as the design is responsive.

3. System Requirements

3.1 Software Requirements

Item	Details
Language	Python 3.11 or newer

Web framework	FastAPI with Uvicorn server
Templates	Jinja2, HTML, CSS, JavaScript
AI library	google-genai (Gemini API)
Database	SQLAlchemy with SQLite (local) or PostgreSQL (Render)
Other libraries	python-dotenv, python-multipart, itsdangerous, psycopg
Hosting	GitHub and Render
Browser	Chrome, Edge, Firefox or any modern browser

3.2 Hardware Requirements

- Computer or laptop with at least 4 GB RAM for development.
- Internet connection to reach the Gemini API and Render.
- Any smartphone or PC with a browser for the users.

4. System Design

4.1 Working Flow

The steps below explain how the application works from the user's side.

- The user registers with email, password, age, weight, fitness goal and intensity, and verifies the mobile number if required.
- After login, the user opens the home page (training card) and presses the generate button.
- The server calls Gemini 2.5 Pro with a prompt that asks for a 7-day plan in JSON format.
- The server calls Gemini 2.5 Flash to get a short nutrition and recovery tip.
- The plan and the tip are saved in the database and shown on the result page.
- If the user gives feedback, the old plan and the feedback are sent to Gemini 2.5 Pro and the updated plan is saved and shown.

4.2 Modules

Module	Work done
auth.py	Password hashing and checking, session handling
routes.py	All pages: login, register, OTP, generate workout, feedback, diet chat
gemini_generator.py	Creates the 7-day workout plan using Gemini 2.5 Pro
gemini_flash_generator.py	Creates the nutrition tip using Gemini 2.5 Flash

updated_plan.py	Revises the plan using the user's feedback
diet_chat.py	Diet chat replies using Gemini 2.5 Flash
password_reset.py	Email OTP for forgot password
verification.py	Mobile OTP (Twilio Verify) and Turnstile bot check
models.py / database.py	Database tables and connection
main.py	Starts the app and gives the /healthz health check

4.3 Database Design

Table	Main columns
users	id, user_id, email, name, age, weight, fitness_goal, workout_intensity, password_hash, mobile_phone, created_at
workout_plans	id, user_id, original_plan, nutrition_tip, feedback, updated_plan, created_at, updated_at
user_feedback	id, user_id, rating, category, message, created_at
diet_chat_messages	id, user_id, role, content, created_at
password_reset_tokens	id, user_id, token_digest, token_salt, attempts, expires_at, used_at
pending_registrations	id, email, phone, name, age, weight, goal, intensity, password_hash, expires_at

4.4 Routes

Route	Method	Purpose
/login	GET/POST	Email and password sign-in
/register	GET/POST	Create account and profile
/forgot-password, /reset-password	GET/POST	Email code and new password
/verify-mobile	GET/POST	SMS code check
/	GET	Member training card
/generate-workout	POST	Generate 7-day plan and tip
/submit-feedback	POST	Update the plan from feedback
/feedback, /feedback-inbox	GET/POST	Send and read member feedback
/diet-chat	GET/POST	Private diet chat
/logout	POST	Sign out

/healthz	GET	Health check for Render

5. Use of Gemini Models

Two Gemini models are used, each for the work that suits it.

- Gemini 2.5 Pro is used to generate the workout plan and the updated plan. The prompt tells the model to act as a strength and conditioning coach and to answer only in JSON with exactly 7 days. Each day has focus, warm_up, main_workout and cooldown.
- Gemini 2.5 Flash is used for the short nutrition tip and the diet chat because it replies faster and these tasks are small.

The response is requested as JSON (response_mime_type = application/json) and then parsed by the server, so the result page can show every day in a clean layout. If the API key is not set or the call fails, the app uses a built-in sample plan and tip. This keeps the website working during a demo or when the internet is slow.

6. Security

- Passwords are stored as salted PBKDF2 hashes and never as plain text.
- Password reset uses a 6-digit email OTP which expires in 30 minutes and is locked after 5 wrong attempts.
- Cloudflare Turnstile is used on login, registration and reset requests to stop bots.
- Twilio Verify is used for mobile OTP during registration and sign-in.
- Secret keys (Gemini key, session secret, SMTP and Twilio details) are saved as environment variables, not in the source code.
- Cookies are marked secure when the site runs on HTTPS.

7. Deployment

The project is stored in a GitHub repository and deployed as a Web Service on Render.

Setting	Value
Runtime	Python
Build command	pip install -r requirements.txt
Start command	uvicorn app.main:app --host 0.0.0.0 --port $PORT
Health check path	/healthz
Environment variables	GOOGLE_API_KEY, SESSION_SECRET, COOKIE_SECURE,

	DATABASE_URL, Turnstile, Twilio and SMTP keys

On the free plan of Render the service sleeps after some time without visitors, so the first page load can take around a minute. Files on the free plan are also temporary, so for permanent user accounts a PostgreSQL database should be connected using the DATABASE_URL variable.

8. Testing

No.	Test case	Expected result
1	Register with valid details	Account is created
2	Login with wrong password	Error message is shown
3	Generate workout with valid API key	7-day plan from Gemini is shown
4	Generate workout without API key	Built-in sample plan is shown
5	Submit feedback	Updated plan is saved and shown
6	Forgot password with correct OTP	New password can be set
7	Wrong OTP five times	Reset code is locked
8	Open /diet-chat without login	User is sent to login page

Fill the actual result column with your own test observations and screenshots before final submission.

9. Limitations

- The plans are general guidance and not medical advice.
- The AI can sometimes give a wrong or unsuitable exercise, so the user must use their own judgement.
- Real AI replies need an internet connection and a valid Gemini API key.
- Free hosting has slow start and temporary storage.

10. Future Enhancements

- Mobile app version for Android and iOS.
- Progress charts for weight and completed workouts.
- Video or image demo for each exercise.
- Tamil and other language support.

- A real trainer login to review AI plans.

11. Conclusion

FitBuddy shows that AI models can be used in a simple and useful way to help normal people start their fitness journey. Using Gemini 2.5 Pro for planning and Gemini 2.5 Flash for quick replies gave a good balance between quality and speed. The project also helped our team learn FastAPI, database design, API integration, web security and cloud deployment.

12. References

- Google Gemini API documentation – ai.google.dev
- FastAPI documentation – fastapi.tiangolo.com
- SQLAlchemy documentation – sqlalchemy.org
- Render documentation – render.com/docs
- Cloudflare Turnstile documentation – developers.cloudflare.com
- Twilio Verify documentation – twilio.com/docs/verify